

ÉCOLE NATIONALE SUPÉRIEURE POLYTECHNIQUE DE YAOUNDÉ

UE : Coder pour la VR, l'XR et l'AR II

RAPPORT DE TRAVAUX PRATIQUES

Calcul Numérique & Rasterisation Logicielle

Stabilité numérique · Algèbre linéaire · Pipeline graphique · Quaternions

Étudiant	NGONO NGUIETSI VANINA SYLLA
Matricule	22P012
Université	ENSPY — Yaoundé
Unité d'enseignement	Coder pour la VR, l'XR et l'AR II
Date de rendu	05 Avril 2026

1. Introduction

Ce rapport présente l'ensemble des travaux pratiques réalisés autour du calcul numérique, de l'algèbre linéaire et de la rasterisation logicielle en C++. L'objectif global est de comprendre les fondements mathématiques et algorithmiques qui sous-tendent un moteur graphique 3D temps réel, dans le contexte de la VR, XR et AR.

Les quatre grands axes abordés sont :

- Précision numérique : flottants IEEE 754, erreurs d'arrondi, algorithmes stables (Kahan, Welford)
- Algèbre linéaire : vecteurs (Vec2/Vec3), opérations fondamentales, orthonormalisation de Gram-Schmidt
- Transformations 3D : matrices 4x4 homogènes, TRS, projection perspective, caméra LookAt
- Pipeline graphique : rasterisation logicielle, tracé de lignes (Bresenham), animation par quaternions et SLERP

Environnement technique

Langage : C++17 | Compilation : g++ avec -O2 | Format image : PPM P6 binaire | Plus de 40 tests unitaires implémentés

2. Précision Numérique

La précision numérique est un enjeu critique en calcul scientifique et en informatique graphique. Les erreurs d'arrondi, si elles ne sont pas maîtrisées, peuvent s'accumuler et conduire à des résultats incorrects ou instables.

2.1 Représentation des flottants IEEE 754

La norme IEEE 754 définit le format de représentation des nombres à virgule flottante. Un float 32 bits se décompose en :

- 1 bit de signe
- 8 bits d'exposant (biais de 127)
- 23 bits de mantisse (fraction)

Certains réels décimaux comme 0.1 ne possèdent pas de représentation exacte en base 2. En base binaire, 0.1 est périodique, de la même façon que 1/3 est périodique en base 10. La machine stocke alors une valeur approchée :

$$0.1_{10} \approx 0.0001\ 1001\ 1001\ 1001\ 1001\ 1001\ 1001\ 1010_2 \text{ (tronqué à 32 bits)}$$

Cette approximation est à l'origine d'erreurs d'arrondi cumulatives lors de calculs itératifs.

Point clé

Ne jamais comparer deux flottants avec == en C++. Toujours utiliser $|a - b| < \text{epsilon}$, où epsilon est adapté au contexte (typiquement $1e-5$ à $1e-7$ pour des float).

2.2 Somme et algorithme de Kahan

La sommation naïve de N termes flottants accumule une erreur proportionnelle à $N \cdot \epsilon$. L'algorithme de Kahan compense cette erreur en maintenant une variable de correction c.

```
// Algorithme de Kahan - Sommation compensée
float kahan_sum(const std::vector<float>& v) {
    float sum = 0.0f, c = 0.0f;
    for (float x : v) {
        float y = x - c;           // Correction appliquée
        float t = sum + y;
        c = (t - sum) - y;         // Capture de l'erreur résiduelle
        sum = t;
    }
    return sum;
}
```

Résultats expérimentaux pour la somme de 10^6 fois 0.1f (valeur théorique : 100 000.0) :

Méthode	Résultat obtenu	Erreur absolue
std::accumulate	~100 032.0	~32.0
Kahan	~100 000.0	< 0.01

2.3 Calcul de variance — Méthode de Welford

Le calcul naïf de la variance (calculer d'abord la moyenne, puis la somme des carrés des écarts) peut produire une annulation catastrophique lorsque les valeurs sont grandes et proches. La méthode en ligne de Welford évite ce problème en mettant à jour la moyenne et la variance en une seule passe :

$$\delta = x_n - \mu_{n-1} \quad ; \quad \mu_n = \mu_{n-1} + \delta/n \quad ; \quad M_n = M_{n-1} + \delta \cdot (x_n - \mu_n)$$

La variance est ensuite obtenue par $\sigma^2 = M_n / (n-1)$. Cette formulation est numériquement stable même pour des grands jeux de données.

2.4 Epsilon machine

L'epsilon machine ε est le plus petit nombre tel que $1.0 + \varepsilon \neq 1.0$ en arithmétique flottante. Sa valeur théorique pour un float 32 bits est :

$$\varepsilon_{machine} \approx 1.19 \times 10^{-7} \quad (\text{soit } 2^{-23})$$

La mesure expérimentale, obtenue par division successive par 2, confirme cette valeur et coïncide avec `std::numeric_limits<float>::epsilon()` de la bibliothèque standard C++.

3. Vecteurs et Algèbre Linéaire

L'algèbre vectorielle est le langage naturel des transformations géométriques 3D. Nous avons implémenté des structures Vec2 et Vec3 en C++ avec toutes les opérations fondamentales.

3.1 Structures Vec2 et Vec3

Les deux structures encapsulent les opérations arithmétiques de base ainsi que les opérations géométriques :

```
struct Vec3 {
    float x, y, z;
    float dot(const Vec3& b) const { return x*b.x + y*b.y + z*b.z; }
    Vec3 cross(const Vec3& b) const {
        return { y*b.z - z*b.y,
                z*b.x - x*b.z,
                x*b.y - y*b.x }; }
    float norm() const { return std::sqrt(dot(*this)); }
    Vec3 normalize() const { float n = norm(); return {x/n, y/n, z/n}; }
};
```

3.2 Produit scalaire et produit vectoriel

Le produit scalaire mesure la similarité directionnelle entre deux vecteurs :

$$a \cdot b = a_x b_x + a_y b_y + a_z b_z = \|a\| \|b\| \cos \theta$$

Le produit vectoriel donne un vecteur orthogonal aux deux opérandes, fondamental pour le calcul des normales :

$$a \times b = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

3.3 Orthonormalisation de Gram-Schmidt

Le procédé de Gram-Schmidt construit une base orthonormée à partir d'un ensemble de vecteurs quelconques. Pour deux vecteurs a et b :

$$u = a / \|a\|$$

$$v' = b - (b \cdot u) \cdot u \rightarrow v = v' / \|v'\|$$

Cette base est utilisée notamment pour construire la matrice de vue (caméra LookAt) à partir des vecteurs right, up et forward.

4. Matrices et Transformations 3D

Les matrices homogènes 4×4 permettent de représenter toutes les transformations affines (translation, rotation, mise à l'échelle) dans un formalisme unifié, facilitant leur composition par simple multiplication matricielle.

4.1 Matrice TRS

Une transformation objet complète est décomposée en Translation (T), Rotation (R) et Scale (S). L'ordre d'application est fondamental :

$$M_{TRS} = T \cdot R \cdot S$$

La décomposition inverse permet d'extraire chaque composante depuis une matrice 4×4 :

- Translation : directement en colonne 4 ($M[0][3]$, $M[1][3]$, $M[2][3]$)
- Scale : norme des 3 premières colonnes de la sous-matrice 3×3
- Rotation : sous-matrice 3×3 normalisée par les facteurs de scale

4.2 Projection perspective

La projection perspective simule la perception naturelle de la profondeur en divisant par la coordonnée Z. Les coordonnées image (u, v) d'un point 3D (X, Y, Z) sont :

$$u = f_x \cdot (X/Z) + c_x \quad v = f_y \cdot (Y/Z) + c_y$$

La focale est dérivée du champ de vision (FOV) :

$$f = 1 / \tan(FOV/2)$$

En pratique, la projection perspective complète s'écrit sous forme matricielle pour permettre la composition avec d'autres transformations :

$$P = \begin{bmatrix} f/aspect & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & -(f+n)/(f-n) & -2fn/(f-n) \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

4.3 Matrice de vue — LookAt

La matrice de vue transforme les coordonnées du monde en coordonnées caméra. Elle est construite à partir de trois paramètres :

- eye : position de la caméra dans le monde
- target : point regardé (centre de la scène)
- up : vecteur haut de référence (typiquement (0,1,0))

```
Mat4 lookAt(Vec3 eye, Vec3 target, Vec3 up) {
    Vec3 fwd = (target - eye).normalize(); // axe Z négatif
    Vec3 rgt = fwd.cross(up).normalize();   // axe X (droite)
    Vec3 uup = rgt.cross(fwd);             // axe Y (haut réel)
    return Mat4 {
        rgt.x, rgt.y, rgt.z, -rgt.dot(eye),

```

```
        uup.x, uup.y, uup.z, -uup.dot(eye),  
        -fwd.x, -fwd.y, -fwd.z, fwd.dot(eye),  
        0,      0,      0,      1  
    };  
}
```

5. Rasterisation Logicielle

La rasterisation est le processus de conversion d'une scène 3D en image 2D pixel par pixel. Nous avons implémenté un pipeline graphique complet en logiciel pur, sans aucune API graphique externe.

5.1 Pipeline graphique

Le pipeline se déroule en quatre étapes successives :

1. Transformation objet : application de la matrice M_{TRS} à chaque sommet
2. Transformation vue : multiplication par la matrice LookAt pour passer en espace caméra
3. Projection : division perspective pour obtenir les coordonnées NDC (Normalized Device Coordinates)
4. Rasterisation écran : conversion NDC $\rightarrow$ pixels et tracé des primitives

5.2 Algorithme de Bresenham

L'algorithme de Bresenham est utilisé pour tracer les arêtes du cube avec une précision entière, sans calcul de virgule flottante :

```
void bresenham(int x0, int y0, int x1, int y1,
               Image& img, Color c) {
    int dx = abs(x1-x0), dy = abs(y1-y0);
    int sx = x0<x1 ? 1:-1, sy = y0<y1 ? 1:-1;
    int err = dx - dy;
    while (true) {
        img.setPixel(x0, y0, c);
        if (x0==x1 && y0==y1) break;
        int e2 = 2*err;
        if (e2 > -dy) { err -= dy; x0 += sx; }
        if (e2 <  dx) { err += dx; y0 += sy; }
    }
}
```

5.3 Format de sortie PPM

Les images sont sauvegardées au format PPM (Portable Pixmap) binaire P6, un format simple lisible par la plupart des visionneuses d'images :

```
P6
<largeur> <hauteur>
255
<données binaires RGB - 3 octets par pixel>
```

5.4 Animation par rotation

Une animation de 10 images est générée en appliquant une rotation incrémentale autour de l'axe Y entre chaque frame. La matrice de rotation élémentaire autour de Y est :

$$R_Y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \end{bmatrix}$$

[0 0 0 1]

Chaque image est sauvegardée sous la forme frame_00.ppm, frame_01.ppm, ..., frame_09.ppm, et peut être assemblée en GIF ou vidéo avec FFmpeg.

6. Quaternions et Animation SLERP

Les quaternions sont une alternative supérieure aux matrices de rotation pour représenter les orientations en 3D, car ils évitent le gimbal lock, interpolent naturellement et sont plus économiques en mémoire (4 flottants vs 9 pour une matrice 3×3).

6.1 Définition et propriétés

Un quaternion unitaire $q = (w, x, y, z)$ encode une rotation d'angle θ autour d'un axe normalisé $n = (n_x, n_y, n_z)$:

$$q = \cos(\theta/2) + \sin(\theta/2) \cdot (n_x i + n_y j + n_z k)$$

Les propriétés fondamentales vérifiées :

- Identité : $q \cdot q^{-1} = 1$ (avec q^{-1} = conjugué si unitaire)
- Application d'une rotation : $v' = q \cdot v \cdot q^*$ (produit de Hamilton)
- Composition : rotation (q_1 puis q_2) = $q_2 \cdot q_1$

Validation implémentée : rotation de (1,0,0) par $q = \text{Quat}(\{0,1,0\}, \pi/2)$ donne (0,0,-1), confirmant la rotation de 90° autour de Y.

6.2 Conversion Quaternion ↔ Matrice

La conversion quaternion → matrice 3×3 est obtenue par développement du produit $q \cdot v \cdot q^*$:

$$R = \begin{bmatrix} 1-2(y^2+z^2) & 2(xy-wz) & 2(xz+wy) \\ 2(xy+wz) & 1-2(x^2+z^2) & 2(yz-wx) \\ 2(xz-wy) & 2(yz+wx) & 1-2(x^2+y^2) \end{bmatrix}$$

La conversion inverse (matrice → quaternion) est testée sur 50 cas aléatoires avec vérification de l'identité aller-retour.

6.3 SLERP vs LERP

L'interpolation entre deux orientations q_0 et q_1 peut se faire par LERP (interpolation linéaire) ou SLERP (interpolation sphérique) :

$$SLERP(q_0, q_1, t) = q_0 \cdot (q_0^* \cdot q_1)^t = \frac{\sin((1-t)\Omega)}{\sin(\Omega)} \cdot q_0 + \frac{\sin(t\Omega)}{\sin(\Omega)} \cdot q_1$$

Méthode	Avantage	Inconvénient
LERP	Rapide, simple	Vitesse angulaire non constante
SLERP	Rotation uniforme, trajectoire géodésique	Légèrement plus coûteux

Une animation de 60 frames a été générée avec SLERP entre deux orientations distinctes d'un cube, démontrant visuellement la fluidité de la rotation.

7. Tests et Validation

Plus de 40 tests unitaires ont été implémentés pour garantir la correction de chaque composant. Les tests couvrent :

Domaine	Nombre de tests	Exemples
Précision numérique	8	<i>Kahan, Welford, epsilon</i>
Opérations vectorielles	12	<i>dot, cross, normalize, Gram-Schmidt</i>
Matrices & inverses	10	<i>TRS, LookAt, projection</i>
Quaternions	8	<i>rotation, conversion, SLERP</i>
Pipeline graphique	5+	<i>projection cube, Bresenham</i>

8. Conclusion

Ce projet de travaux pratiques a permis d'explorer en profondeur les fondements mathématiques et algorithmiques du calcul numérique et du rendu graphique 3D. Les principaux enseignements sont :

- Stabilité numérique : les algorithmes de Kahan et Welford montrent qu'un simple changement d'ordre d'opérations peut réduire l'erreur de plusieurs ordres de grandeur. Ce principe est fondamental pour tout calcul embarqué temps réel.
- Algèbre linéaire appliquée : les vecteurs, matrices et quaternions ne sont pas de simples abstractions mathématiques — ils sont le langage de tout moteur graphique moderne. La compréhension de leurs propriétés est indispensable pour déboguer et optimiser.
- Pipeline graphique : implémenter un rasteriseur logiciel complet, même simple, permet de comprendre ce que font OpenGL, Vulkan et les APIs graphiques modernes sous le capot. Chaque étape (transformation, projection, rasterisation) a sa raison d'être précise.
- Quaternions et animation : SLERP offre une interpolation de rotation nettement supérieure à LERP, avec un coût computationnel modeste. C'est la norme dans les moteurs de jeu comme Unity ou Unreal Engine.

Perspectives

Extensions possibles : implémentation d'un z-buffer pour la gestion des occlusions, ajout d'un modèle d'illumination (Phong, Blinn-Phong), export au format BMP ou PNG, et intégration d'un chargeur de fichiers OBJ pour afficher des modèles 3D arbitraires.